

Dokumentácia k práci na predmetoch tímového vývoja softvéru
na FMFI UK v akademickom roku 2025/2026

Mutation-based Testing

Členovia tímu:

Ondrej Babinský	<i>Team Leader</i>
Richard Macko	<i>Full-Stack Developer</i>
Matúš Ratkovský	<i>Full-Stack Developer</i>
Steven Gavlák	<i>Full-Stack Developer</i>
Jozef Blaško	<i>Full-Stack Developer</i>

Cvičiaci: Martin Marko

GitHub:

<https://github.com/Bolest-oci/mutation-testing>

January 2026

Obsah

1	Dokumentácia k riadeniu projektu	1
1.1	Šprint 1 (20. 10. 2025 – 3. 11. 2025)	1
1.2	Šprint 2 (3. 11. 2025 – 17. 11. 2025)	2
1.3	Šprint 3 (17. 11. 2025 – 1. 12. 2025)	4
1.4	Šprint 4 (1. 12. 2025 – 15. 12. 2025)	5
1.5	Šprint 5 (1. 01. 2026 – 18. 01. 2026)	7
2	Analytická dokumentácia	9
2.1	Úvod	9
2.2	Use Case 01 – Project Initialization & Framework Setup	10
2.2.1	Inštalácia upraveného forku Cosmic Ray	10
2.2.2	Validácia a Konfigurácia	11
2.3	Use Case 02 - Automated custom mutators integration	12
2.4	Use Case 03 – Run Targeted Mutation Test Runs	13
2.4.1	Sekvenčný diagram	14
2.5	Use Case 04 – Generate and Interpret Mutation Testing Results	15
2.6	Doplňujúce technické úlohy a rozšírenia projektu	17
2.6.1	Porovnanie mutation testing frameworkov	18
2.6.2	Odhalenie chýb v knižnici <code>validators</code> pomocou mutation testingu	20
2.6.3	Úprava HTML reportov v Cosmic Ray	21
2.6.4	Zabezpečenie behu RooCode v PyCharme	22
2.6.5	Rozšírenie kontextu mutácií o názov funkcie	22
3	Technická dokumentácia	24
3.1	Použité technológie	24
3.1.1	Programovací jazyk	24
3.1.2	Frameworky pre mutation testing	24
3.1.3	Vývojové prostredia	24
3.1.4	Roo Code	24
3.1.5	Databáza	24
3.1.6	Správa verzií	25
3.2	Deployment diagram	25
3.3	Dátový model	26
4	Zaujímavé fragmenty implementácie	27
4.1	Úprava HTML reportov v Cosmic Ray: skrytie preskočených mutácií	28
4.2	Rozšírenie kontextu mutácií o názov funkcie	28
4.3	Automatická AI analýza preživších mutácií	30
4.3.1	Ukážka časti <code>roo code</code> scenára pre vyhodnotenie mutation testingu	30
4.3.2	Princíp AI hodnotenia	30
4.3.3	Výstup AI analýzy	31
4.4	Implementácia custom mutátora	32
5	Inštalačná príručka	33
5.1	Príprava pracovného priestoru	33
5.2	Inštalácia nástroja Cosmic Ray	33
5.3	Synchronizácia a overenie	33

1 Dokumentácia k riadeniu projektu

Táto sekcia popisuje priebeh riešenia projektu z pohľadu jeho riadenia. Projekt bol realizovaný iteratívnym spôsobom pomocou týždenných šprintov, v rámci ktorých boli plánované a riešené jednotlivé úlohy (stories). Každý šprint obsahuje prehľad realizovaných stories a stručnú retrospektívu, ktorá identifikuje problémy a navrhuje zlepšenia do ďalšieho vývoja.

1.1 Šprint 1 (20. 10. 2025 – 3. 11. 2025)

Začiatok šprintu: 20. 10. 2025

Koniec šprintu: 3. 11. 2025

Stories

Názov	Typ	Story points	Riešitelia	Stav akceptácie
Inicializácia repozitára a projektovej organizácie	Infrastructure	2	Ondrej Babinský	Prijatá
Zber a zdieľanie zdrojov k mutation-based testingu	Research	5	Všetci	Prijatá s pripomienkou
Výber a príprava testovacieho prostredia pre experimenty	Infrastructure	2	Všetci	Prijatá
AI-assisted analýza mutation-based testingu a zdieľanie poznatkov	Research	2	Všetci	Prijatá
Inštalácia a rozbehovanie RooCode agenta vo VS Code	Tooling	2	Všetci	Prijatá

Retrospektíva

Problém	Action item(s)
Úvodné úlohy boli roztrúsené medzi viacero miest (poznámky, chaty) a hrozila strata kontextu.	Zaviest' jednotné pravidlo: všetky poznatky a priebežné výsledky ukladať do repozitára a do dohodnutých priečinkov (/research, /ai-insights).
Niektoré úlohy boli príliš všeobecné, čo sťažovalo odhad náročnosti a plánovanie šprintu.	Pri plánovaní rozbiť veľké úlohy na menšie, merateľné stories (jasný výstup: súbor, dokument, skript, report).

1.2 Šprint 2 (3. 11. 2025 – 17. 11. 2025)

Začiatok šprintu: 3. 11. 2025

Koniec šprintu: 17. 11. 2025

Stories

Názov	Typ	Story points	Riešitelia	Stav akceptácie
Preskúmanie AI asistentov pre PyCharm (alternatívy k RooCode/Cline)	Research	2	Ondrej Babinský	Prijatá
Porovnanie mutačných frameworkov	Research	2	Ondrej Babinský	Prijatá
Experiment s MutMut na referenčnom repozitári	Experimentation	3	Steven Gavlák	Prijatá
Preskúmanie a testovanie MutPy frameworku	Experimentation	3	Matúš Ratkovský	Prijatá s pripomienkou
Preskúmanie a testovanie XMutant frameworku	Experimentation	2	Ondrej Babinský	Prijatá
Preskúmanie a testovanie Cosmic Ray frameworku	Experimentation	5	Richard Macko	Prijatá
Testovanie OpenAI modelov v RooCode	Tooling	2	Steven Gavlák	Prijatá
Analýza náročnosti tvorby custom mutanta v rôznych frameworkoch	Experimentation	3	Jozef Blaško	Prijatá

Retrospektíva

Problém	Action item(s)
Výskumné úlohy boli často vykonávané individuálne bez spoločného zdieľania záverov, čo spôsobovalo duplicitu práce.	Zdieľať hlavné poznatky cez tímový meeting počas každého šprintu.
Porovnávanie rôznych frameworkov bolo náročné bez definovanej sady kritérií.	Vytvoriť spoločnú metriku porovnávania frameworkov (kritériá: výkon, počet mutátorov, selektívne mutovanie, reporting, kompatibilita).
Niektoré experimenty (MutMut, Cosmic Ray, XMutant) mali nejasné výstupy a nebolo jasné, čo považovať za „hotové“.	Zaviesť minimálnu špecifikáciu výstupu pre experiment: spustiteľný príklad, screenshot reportu, opis limitácií, odporúčanie do ďalšej fázy projektu.

1.3 Šprint 3 (17. 11. 2025 – 1. 12. 2025)

Začiatok šprintu: 17. 11. 2025

Koniec šprintu: 1. 12. 2025

Stories

Názov	Typ	Story points	Riešitelia	Stav akceptácie
Vytvorenie vlastného Custom Mutanta v Cosmic Ray	Experimentation	5	Matúš Ratkovský	Prijatá
Automatizácia spracovania survived mutácií v Cosmic Ray	Enhancement	3	Richard Macko	Prijatá
Vytvorenie tabuľky porovnávajúcej mutation frameworky	Documentation	3	Ondrej Babinský	Vrátená na prepracovanie
Zabezpečenie behu RooCode v PyCharm pomocou RunVSAgent a vytvorenie dokumentácie	Tooling	4	Ondrej Babinský	Prijatá s pripomienkou
Analýza možnosti použitia MutMut mutátorov v Cosmic Ray	Research	3	Jozef Blaško	Prijatá s pripomienkou
Implementácia String mutačného operátora pre Cosmic Ray	Feature	3	Jozef Blaško	Prijatá

Retrospektíva

Problém	Action item(s)
Práca s vlastnými mutátormi mala vysokú mieru technickej neistoty, čo spôsobovalo dlhšie hľadanie správneho postupu.	Zaviesť interný „dev log“, kde počas experimentov budú členovia tímu zapisovať všetky kroky a riešenia, aby sa predišlo opakovaniu slepých uličiek.
Integrácia RooCode v PyCharme bola komplikovanejšia než očakávané kvôli potrebe RunVSAgent a rozdielom oproti VS Code.	Do budúcnosti pripraviť stručný „compatibility checklist“ pre nové AI nástroje, aby sa rýchlo zistilo, čo sa dá integrovať a čo nie.

1.4 Šprint 4 (1. 12. 2025 – 15. 12. 2025)

Začiatok šprintu: 1. 12. 2025

Koniec šprintu: 15. 12. 2025

Stories

Názov	Typ	Story points	Riešitelia	Stav akceptácie
Vytvorenie Custom Mutanta v Cosmic Ray	Feature	3	Matúš Ratkovský	Prijatá
Spracovanie survived mutácií a získavanie <i>function_name</i>	Enhancement	3	Richard Macko	Prijatá
Doplnenie zdrojov do tabuľky porovnania frameworkov	Documentation	2	Ondrej Babinský	Prijatá
Využitie AI pri analýze výsledkov mutácií	Research	3	Steven Gavlák	Prijatá s pripomienkou
Objavenie bugu v knižnici <i>validators</i> pomocou mutation testingu	Bug	2	Ondrej Babinský	Prijatá
Oprava chyby v prenose dát o pozícii mutácie a doplnenie <i>function_name</i> do HTML reportu	Bugfix	5	Richard Macko	Prijatá
Vytvorenie návodu na prácu s Cosmic Ray dumpom pomocou jq	Documentation	2	Richard Macko	Prijatá
Integrácia úloh pre rozšírený HTML report a jednoduchšiu konfiguráciu <i>setup.py</i>	Task	3	Steven Gavlák	Prijatá
Implementácia tuple custom mutanta v Cosmic Ray	Feature	5	Matúš Ratkovský	Prijatá

Retrospektíva

Problém	Action item(s)
Komplexné úpravy v Cosmic Ray (nové mutátory, úpravy dumpov, HTML reporty) boli časovo náročné a viac prepojené, než sa očakávalo.	Zaviesť priebežné code review k väčším zásahom do nástroja, aby sa problémy odhalili skôr.
AI analýza výsledkov bola obmedzená kvalitou dumpov, ktoré bolo najprv nutné rozšíriť.	Pred implementáciou AI krokov skontrolovať formát a úplnosť dát, ktoré majú AI asistenti spracovávať.
Pri práci s externými knižnicami (napr. validators) neexistuje garancia správneho mappingu mutácií na pôvodný kód.	Testovať mutácie aj na menších knižniciach a priebežne validovať korektnosť vypočítaných pozícií mutácií.

1.5 Šprint 5 (1. 01. 2026 – 18. 01. 2026)

Začiatok šprintu: 1. 01. 2026

Koniec šprintu: 18. 01. 2026

Stories

Názov	Typ	Story points	Riešitelia	Stav akceptácie
Analýza Git-based diff filtering a relevance filtering	Research	2	Ondrej Babinský	Prijatá
Vytvorenie promptu na automatické mutovanie nad vybraným kódom	Tooling	3	Jozef Blaško	Prijatá
Analýza fungovania cr-filter-git	Research	1	Ondrej Babinský	Prijatá
Zlepšenie HTML reportu Cosmic-Ray odstránením šumu zo SKIPPED mutácií	Feature	3	Ondrej Babinský	Prijatá
Vytvorenie RooCode príkazu na automatizáciu krokov pri práci s custom mutátormi	Tooling	3	Steven Gavlák	Prijatá
User stories a use-case scenáre pre AI agenta RooCode na integráciu custom mutátorov	Analysis	5	Matúš Ratkovský	Prijatá
User stories a use-case scenáre pre pomoc pri opravách a rozširovaní existujúcich testov	Analysis	5	Jozef Blaško	Prijatá
Pridanie príkladov konfiguračných súborov pre Cosmic-Ray	Documentation	2	Ondrej Babinský	Prijatá
User stories a use-case scenáre pre cieľné spúšťanie mutation testov	Analysis	5	Ondrej Babinský	Prijatá s pripomienkou
Úprava a rozšírenie use-case scenárov pre spúšťanie mutation testov	Documentation	2	Richard Macko	Prijatá
Pridanie .patch pre Cosmic-Ray s podporou function_name	Development	3	Richard Macko	Prijatá

Retrospektíva

Problém	Action item(s)
Zvyšujúci sa počet optimalizácií (filtering, AI, custom mutátory) zvyšoval komplexitu workflow.	Navrhnuť jednotný, zdokumentovaný workflow pre spúšťanie mutation testingu s využitím AI a filtrov.
Závislosť na úpravách externého nástroja (Cosmic-Ray) komplikovala opakovateľnosť experimentov.	Automatizovať aplikovanie úprav a jasne ich zdokumentovať.
Use-case scenáre vznikali postupne a neboli vždy konzistentné medzi členmi tímu.	Zjednotiť šablónu pre písanie user stories a use-case scenárov pre ďalšie projekty.

2 Analytická dokumentácia

2.1 Úvod

Cieľom projektu bolo preskúmať a zdokumentovať moderný prístup k testovaniu kvality softvéru v jazyku Python so zameraním na techniky mutation testingu. Zadanie projektu sa sústreďovalo na analýzu existujúcich nástrojov pre mutation testing, ich praktické využitie a možnosti rozšírenia v reálnych softvérových projektoch. Úlohou tímu nebolo vytvoriť nový mutation testing framework, ale systematicky preskúmať dostupné riešenia, identifikovať ich silné a slabé stránky a navrhnúť spôsob, ako ich efektívne používať a rozširovať v rámci bežného vývojového procesu s použitím AI agentov.

V rámci riešenia projektu sa tím venoval návrhu a realizácii user stories a use-case scenárov, ktoré reflektujú reálne potreby vývojárov pracujúcich s Python projektmi. Hlavným cieľom bolo ukázať praktické využitie mutation testing techník, preskúmať dostupné frameworky a knižnice pre Python a posunúť sa od náhodného (random) mutation testovania k targeted prístupu. Dôležitou súčasťou riešenia bolo využitie umelej inteligencie, konkrétne AI asistenta RooCode, ktorý slúžil ako riadiaci prvok celého workflowu na základe špecificky navrhnutých promptov.

Jedným z výsledkov projektu sú tabuľky, ktoré umožňujú používateľovi lepšie sa rozhodnúť pri výbere mutation testing frameworku pre konkrétny Python projekt (časť 2.6.1). Pomocou pripravených, na mieru navrhnutých promptov pre AI agenta RooCode dokáže používateľ realizovať jednotlivé kroky mutation testovania bez potreby detailnej znalosti interných mechanizmov použitých nástrojov. V projekte nevznikol samostatný automatizovaný systém ani skript, ktorý by tieto kroky vykonával; namiesto toho boli vytvorené opakovateľné promptové postupy, ktoré umožňujú RooCode agentovi viesť používateľa procesom nastavenia projektu, výberu frameworku a spustenia mutation testov formou asistovaného workflowu.

Navrhnuté prompty pokrývajú celý proces práce s mutation testingom – od úvodného nastavenia projektu a samotného frameworku (časť 2.2), cez integráciu vlastných alebo externých mutačných operátorov (2.3), až po spúšťanie cielených mutation testov nad vybranými časťami kódu (2.4) a následnú prácu s výsledkami (2.5). Cielené mutation testy je možné aplikovať napríklad len na zmenené riadky alebo súbory, pričom AI agent vyberá vhodné mutačné operátory, určuje relevantné testy a dokáže ignorovať nízkohodnotné časti kódu na základe lintingu alebo code coverage. Súčasťou workflowu je aj vyhodnocovanie preživších mutácií – agent vie identifikovať slabé miesta testovacej sady a navrhovať úpravy, ktoré zlepšujú jej efektivitu bez zbytočného zvyšovania rozsahu testov. Výsledkom je promptmi riadený proces, ktorý umožňuje vývojárovi vykonávať nastavenie, spúšťanie aj interpretáciu mutation testovania zjednodušeným spôsobom.

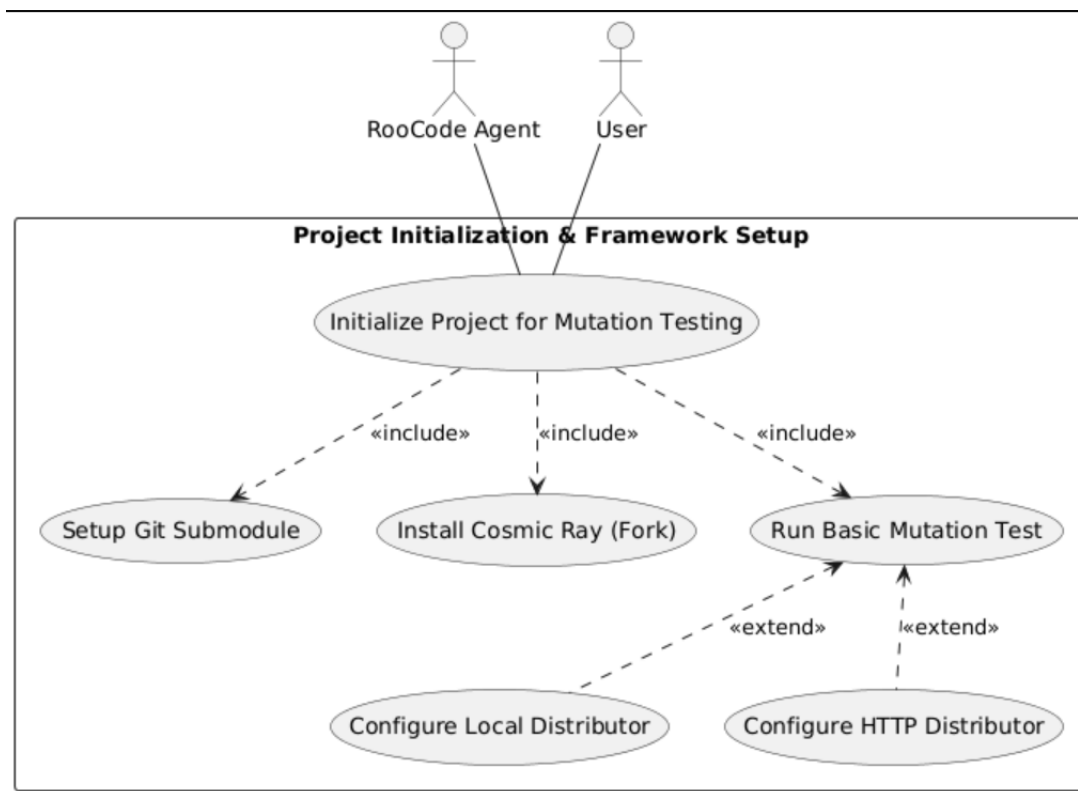
Počas riešenia projektu boli zároveň realizované viaceré praktické úpravy a vylepšenia existujúceho frameworku Cosmic Ray s cieľom zlepšiť používateľskú skúsenosť a praktickú použiteľnosť nástroja. Medzi tieto úpravy patrí napríklad rozšírenie reportov o informáciu `function_name` (časť 4.2) pre každú vykonanú mutáciu, čo umožňuje presnejšie mapovanie mutácií na konkrétne časti kódu, alebo pridanie konfiguračných možností a CLI prepínačov na obmedzenie zobrazovania preskočených mutácií, čím sa znižuje vizuálny šum v reportoch (časť 2.6.3).

2.2 Use Case 01 – Project Initialization & Framework Setup

Táto sekcia analyzuje vstupnú fázu celého riešenia, ktorá je nevyhnutnou prerekvizitou pre ďalšie mutačné testovanie. Proces inicializácie je rozdelený do troch logických krokov, ktoré na seba priamo nadväzujú:

1. **Use Case 01 - Project Setup:** Zabezpečuje izoláciu prostredia. Testovaný projekt je pridaný ako git submodule do adresára `repos/` a je vytvorená dedikovaná vetva (branch), aby sa oddelil testovací proces od produkčného kódu.
2. **Use Case 01.1 - Install Cosmic Ray:** Rieši integráciu nášho upraveného forku nástroja do projektu. Agent deteguje používaný manažér závislostí a vykoná inštaláciu priamo z GitHubu.
3. **Use Case 01.2 - Run Basic Mutation Test:** Slúži na validáciu prostredia a konfigurácie pred spustením náročnejších scenárov.

Vzťahy a závislosti medzi týmito krokmi sú znázornené na nasledujúcom diagrame.



Obr. 1: Use Case diagram pre fázu Project Setup

2.2.1 Inštalácia upraveného forku Cosmic Ray

V rámci kroku UC 01.1 [7] sa zabezpečuje integrácia nášho forku Cosmic Ray do vývojového prostredia projektu. Agent automaticky analyzuje štruktúru projektu a identifikuje nástroje ako `uv`, `poetry`, `pdm` alebo `pip`. Následne zvolí adekvátny príkaz na inštaláciu nášho forku cez protokol `git+https`. Tento postup garantuje, že prostredie bude obsahovať rozšírené funkcionality.

2.2.2 Validácia a Konfigurácia

Scenár UC 01.2 [8] rieši prvé reálne spustenie mutačného testovania. Jeho cieľom je potvrdiť, že projekt je v stave, kedy je možné bezpečne vykonávať mutácie, a že nástroje sú správne nastavené.

Proces pozostáva z nasledujúcich krokov:

1. **Overenie stavu projektu:** Pred začiatkom mutačného testovania je nevyhnutné overiť, či je aktuálna verzia projektu stabilná. Spustí sa preto štandardná testovacia sada (napr. `pytest`), aby sa zabezpečilo, že všetky existujúce testy prechádzajú a východiskový stav je bez chýb.
2. **Príprava konfigurácie:** Pre vytvorenie konfiguračného súboru `cosmic-ray.toml` má používateľ na výber z dvoch možností: buď nechá konfiguráciu vygenerovať agentom, alebo potrebné parametre zadá manuálne. V tomto kroku sa zároveň volí spôsob vykonávania testov – lokálne (UC 01.2.1 [9]) pre jednoduchšie použitie alebo pomocou HTTP distribútora (UC 01.2.2 [10]) pre paralelné spracovanie.
3. **Spustenie analýzy:** Proces začína inicializáciou relácie, ktorá pripraví databázu pre sledovanie mutantov. Následne sa spustí samotné vykonávanie mutačných testov, čím sa overí schopnosť systému generovať a vyhodnocovať mutantov v pripravenom prostredí.

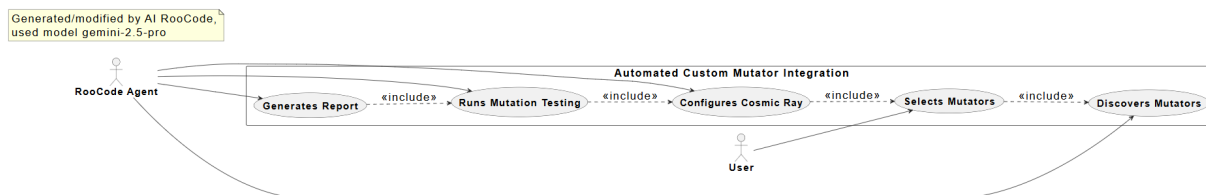
Dynamický priebeh tejto validácie zobrazuje sekvenčný diagram nižšie.



Obr. 2: Sekvenčný diagram pre scenár Run Basic Mutation Test.

2.3 Use Case 02 - Automated custom mutators integration

Táto časť popisuje use-case scenár pre automatickú integráciu custom mutátorov s použitím AI asistenta RooCode. V diagrame sú aktéri AI agent RooCode a používateľ. Hlavným cieľom je zjednodušiť a zefektívniť integráciu custom mutátorov do existujúceho projektu. AI asistent RooCode najprv prehľadá projekt, aby našiel všetky implementované custom mutátory a následne sa používateľa spýta, ktoré z nich chce integrovať. Používateľ si vyberie jeden alebo viac mutátorov. RooCode následne: integruje vybrané custom mutátory, konfiguruje Cosmic Ray, spustí mutačné testovanie len na vybraných custom mutátorov a vygeneruje výsledky mutačného testovania.



Obr. 3: Use case diagram pre scenár Custom Mutators Integration.

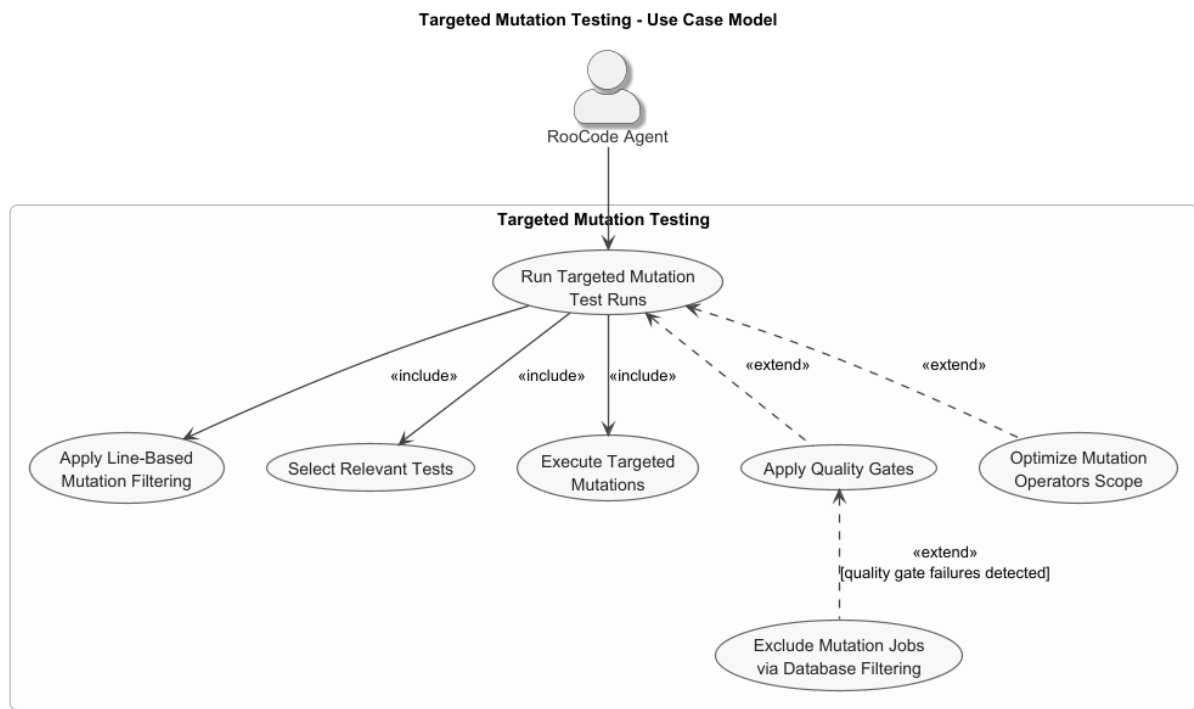
2.4 Use Case 03 – Run Targeted Mutation Test Runs

Táto časť popisuje jeden z riešených use-case scenárov, konkrétne *Run Targeted Mutation Test Runs*.

V diagrame prípadov použitia je aktérom AI agent RooCode, ktorý riadi celý workflow na základe špecificky pripravených `.md` promptov vytvorených v rámci tohto projektu [5]. Tieto prompty definujú jednotlivé kroky procesu (detekcia zmien, aplikácia filtrov, výber operátorov, výber testov, vykonanie mutácií) a umožňujú agentovi realizovať celý scenár bez manuálneho zásahu do konfigurácie frameworku zo strany používateľa.

Hlavným cieľom use casu je vykonať mutation testovanie iba nad relevantnými časťami kódu - teda nad úsekmi, ktoré sa zmenili alebo ktoré sú označené ako relevantné z hľadiska kvality testov. Tento prístup výrazne skracuje čas behu testov a eliminuje nepotrebné mutácie, pričom zachováva kvalitu výsledkov. Súčasťou scenára sú line-based filtre vychádzajúce z git diff, výber testovacích súborov, spustenie mutácií a voliteľné rozšírenia, ako napríklad quality gates alebo optimalizácia rozsahu mutačných operátorov.

Diagram znázorňuje povinné aj nepovinné kroky procesu. Prípady použitia označené ako *include* predstavujú pevné kroky, ktoré sa vždy vykonajú. Vetvy označené ako *extend* predstavujú voliteľné kroky, ktoré sa aktivujú len vtedy, ak ich používateľ nezakáže alebo ak sú dáta v projekte dostupné (napríklad linting alebo coverage informácie). Rovnako aj *Exclude Mutation Jobs via Database Filtering* sa vykoná len v prípade, že quality gates identifikujú problém; inak systém túto časť scenára preskočí.

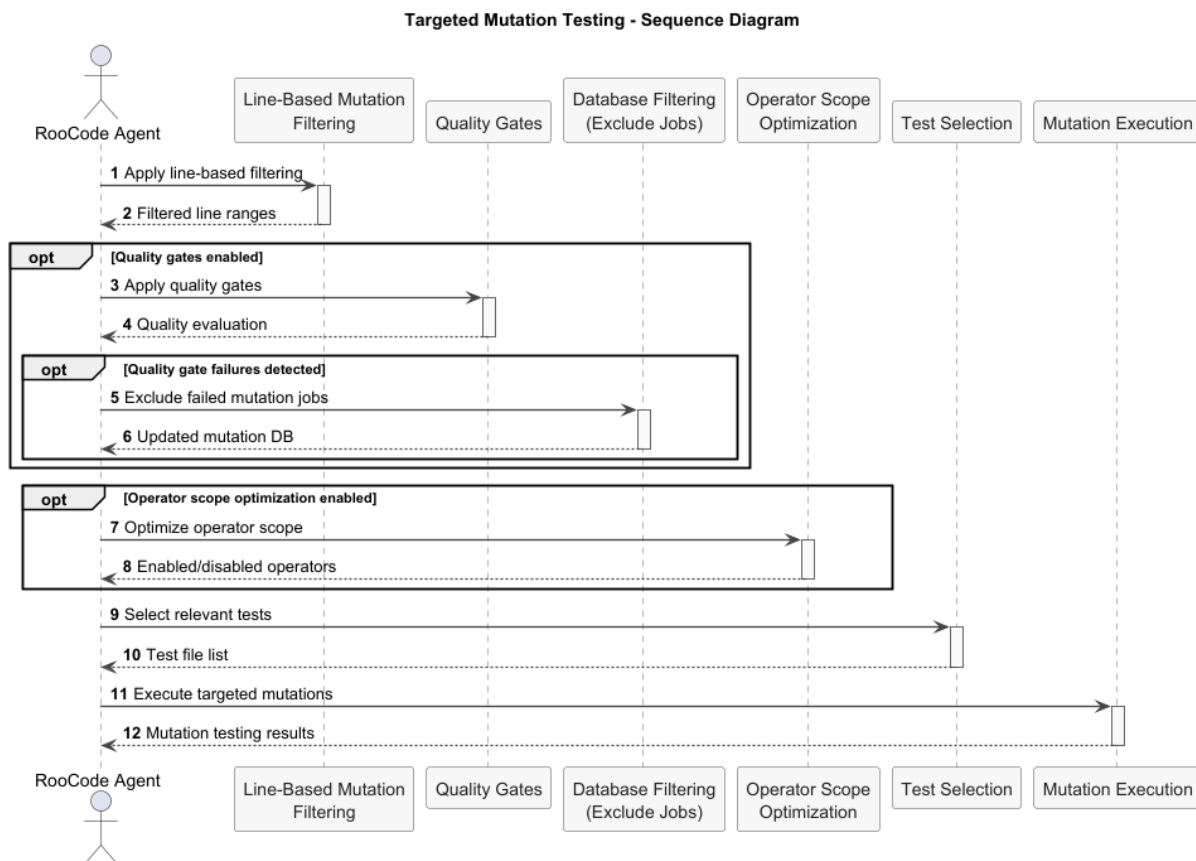


Obr. 4: Use case diagram pre scenár Run Targeted Mutation Test Runs.

Tento use case demonštruje hlavný princíp projektu - spojenie existujúcich mutation testing frameworkov s AI-riadeným rozhodovaním. Pripravené .md prompty poskytujú formalizovaný pracovný postup, vďaka ktorému môže RooCode agent vykonávať ciele mutation testy bez rozsiahleho manuálneho nastavovania.

2.4.1 Sekvenčný diagram

Nasledujúci sekvenčný diagram dopĺňa predošlý use-case diagram a znázorňuje dynamický priebeh scenára *Run Targeted Mutation Test Runs*. Diagram zachytáva jednotlivé kroky, ktoré agent RooCode vykonáva pri ciele mutation testovaní, vrátane voliteľných vetiev (quality gates, database filtering, optimalizácia operátorov). Tento pohľad umožňuje presnejšie pochopiť poradie operácií a interakciu agenta s jednotlivými časťami procesu.



Obr. 5: Sekvenčný diagram pre scenár Run Targeted Mutation Test Runs.

2.5 Use Case 04 – Generate and Interpret Mutation Testing Results

Táto sekcia popisuje use-case scenár zameraný na generovanie a interpretáciu výsledkov mutačného testovania. Ide o hlavný analytický scenár projektu, ktorého cieľom je spracovať výsledky mutation testovania tak, aby z nich vývojár získal prakticky využiteľné informácie o kvalite existujúcej testovacej sady.

Scenár nadväzuje na úspešne vykonané mutačné testovanie a predpokladá, že pred samotnou analýzou bol projekt skontrolovaný pomocou nástrojov pre linting, statickú analýzu a prípadne aj analýzu pokrytia kódu testami. Tieto kroky slúžia na odstránenie irelevantného alebo netestovateľného kódu, aby výsledky mutačného testovania neboli skreslené.

Hlavným aktérom scenára je AI agent RooCode, ktorý pracuje s databázovou reláciou mutačného testovania (`session.sqlite`), projektovým kódom a výstupmi nástroja Cosmic Ray. Cieľom scenára je identifikovať mutácie, ktoré prežili existujúce testy, a na ich základe odhaliť slabé miesta v testovacej sade alebo v samotnej implementácii kódu.

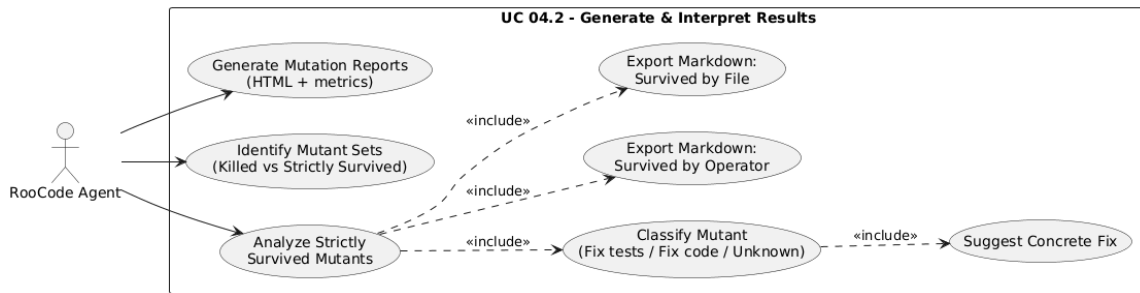
Na začiatku scenára agent vygeneruje HTML report z databázovej relácie mutačného testovania a získa základné metriky, ako je počet zabitých a prežitých mutácií alebo výsledné mutation score. Následne získa zoznam všetkých vykonaných mutácií a rozdelí ich podľa výsledku ich vykonania.

Z ďalšej analýzy sú vylúčené mutácie, ktoré neposkytujú relevantnú informáciu o kvalite testovacej sady, napríklad ekvivalentné mutácie, mutácie bez pokrytia testami alebo

technicky zlyhané mutácie. Pozornosť je tak sústredená výhradne na mutácie, ktoré prežili napriek existujúcim testom.

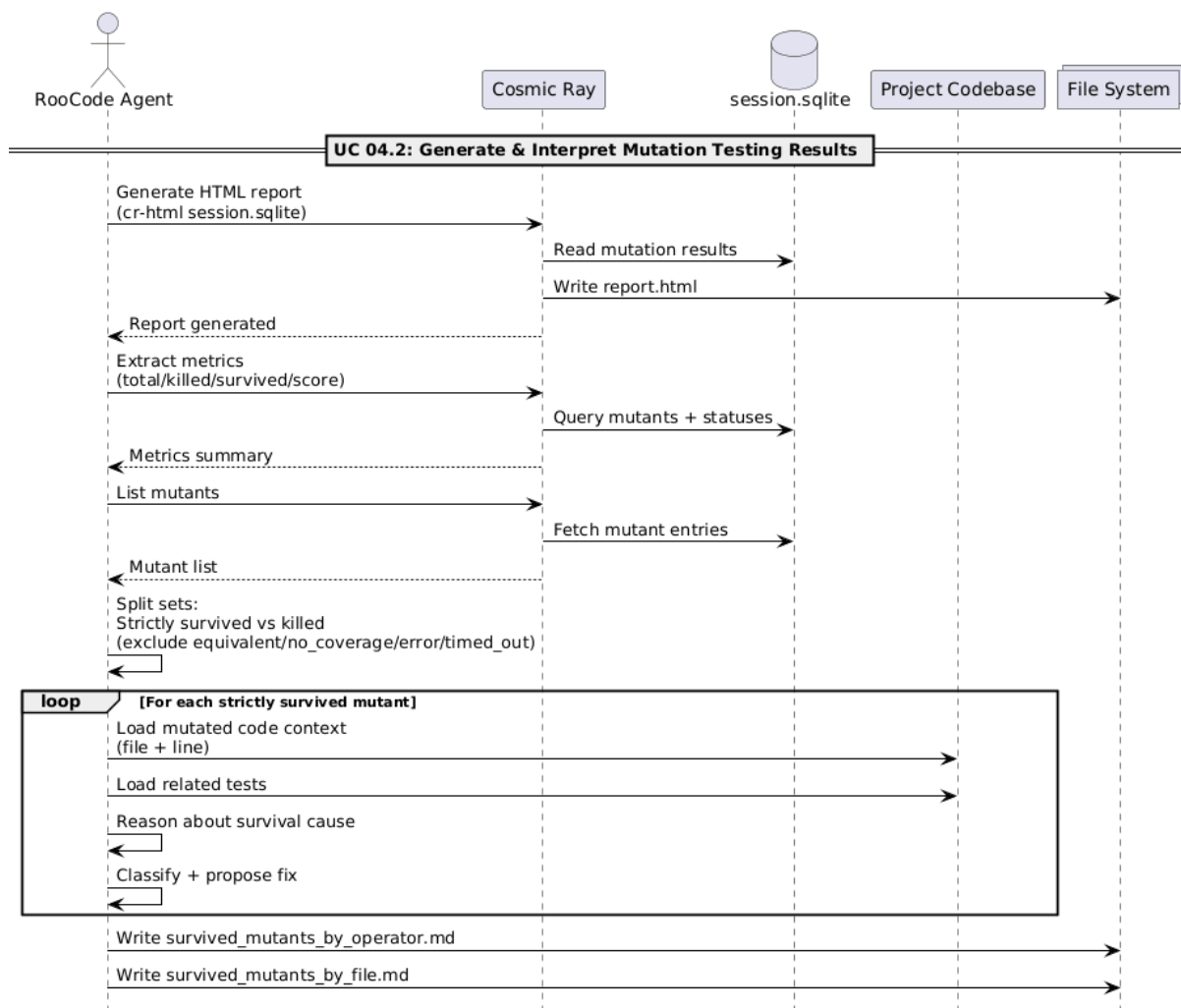
Pre každý striktné prežitý mutant agent analyzuje príslušný úsek produkčného kódu a súvisiace testy. Na základe tejto analýzy sa snaží určiť, či je príčinou prežitia nedostatkový testovací prípad, problematická implementácia kódu, alebo či nie je možné príčinu jednoznačne určiť. Súčasťou výstupu je aj návrh konkrétnej úpravy, napríklad doplnenie testu alebo zmenu existujúcej implementácie.

Vzťahy medzi jednotlivými krokmi scenára sú znázornené na use case diagrame nižšie.



Obr. 6: Use case diagram pre scenár Generate and Interpret Mutation Testing Results.

Sekvenčný diagram znázorňuje poradie jednotlivých krokov, ktoré agent RooCode vykonáva počas spracovania výsledkov mutačného testovania, od generovania reportu až po analýzu striktné prežitých mutácií.



Obr. 7: Sekvenčný diagram pre scenár Generate and Interpret Mutation Testing Results.

2.6 Doplnujúce technické úlohy a rozšírenia projektu

Popri návrhu use-case scenárov a AI workflowu sme realizovali aj viacero doplnkových technických prác, ako porovnanie existujúcich mutation testing nástrojov, testovanie ich správania na reálnych projektoch, hľadanie chýb v používaných Python knižniciach pomocou mutačných testov a implementáciu úprav, ktoré zlepšili celkové praktické používanie Cosmic Ray.

HTML report obsahuje zoznam všetkých spustených mutantov vrátane tých, ktoré prežili (surviving mutants). Prežitie mutantu znamená, že test ho nezachytil, čo poskytuje cennú informáciu o slabých miestach v testovacej sade. Agent následne získa zoznam všetkých vykonaných mutácií a rozdelí ich podľa výsledku vykonania. Z ďalšej analýzy sú vylúčené mutácie, ktoré neposkytujú relevantnú informáciu, ako sú ekvivalentné mutácie, mutácie bez pokrytia testami alebo technicky zlyhané mutácie. Pozornosť je sústredená na mutácie, ktoré prežili napriek existujúcim testom.

Pre každý striktné prežitý mutant agent analyzuje príslušný úsek produkčného kódu a súvisiace testy. Na základe tejto analýzy sa RooCode snaží určiť, či je príčinou prežitia nedostatočný testovací prípad, problematická implementácia kódu alebo iný dôvod, a následne vytvoriť nový test alebo doplniť existujúci, aby pri ďalšom spustení testov mutácia neprežila. V procese sa dobre využijú aj rôzne custom mutátory, ktoré môžu odhaliť

chyby, ktoré predtým neboli zachytené.

Výstupom scenára je aktualizovaná testovacia sada s novými alebo vylepšenými testami, ktorá znižuje počet prežitých mutácií, zvyšuje mutation score, zlepšuje pokrytie testami a odhaľuje slabé miesta v existujúcich testoch aj v samotnom kóde. Tento proces umožňuje efektívne využitie výsledkov mutačného testovania na zlepšenie kvality softvéru a zvýšenie spoľahlivosti testovacej sady.

2.6.1 Porovnanie mutation testing frameworkov

Projekt zahŕňal preskúmanie dostupných mutation testing frameworkov pre Python s cieľom porovnať ich vlastnosti, podporované mutačné operátory, architektúru a vhodnosť pre cielené, workflow-riadené mutačné testovanie v kombinácii s AI agentom. Súčasťou analýzy bolo aj overenie správania jednotlivých nástrojov na reálnych projektoch a posúdenie ich praktickej použiteľnosti. Súhrnná porovnávacia tabuľka je dostupná v repozitári projektu [4].

Výskum zahŕňal porovnanie šiestich relevantných nástrojov (Cosmic Ray, MutMut, MutPy, XMutant, Mutatest a Poodle). Porovnávané boli najmä tieto oblasti:

- **kompatibilita:** podporované verzie Pythonu, operačné systémy a testovacie frameworky,
- **funkcie:** podpora selective mutating a paralelného spracovania,
- **reporting:** formáty výstupov a možnosť pracovať s databázou mutácií,
- **rozšíriteľnosť:** dostupnosť custom mutátorov a AST parserov,
- **údržba:** aktivita komunity a dátum posledného update.

Druhá tabuľka sumarizuje, aké základné typy mutácií nami vybrané frameworky podporujú. Tento prehľad umožňuje posúdiť, aký široký rozsah chýb daný nástroj dokáže detekovať, a tým aj to, nakoľko je vhodný pre analýzu kvality testovacej sady.

	Programming Language	Language version requirements	Last Update	OS	Test Runners	Custom mutators
Cosmic Ray	Python	Python 3.xx	2025	Windows, Linux, MacOS	All relevant	Yes
MutMut	Python	Python >=3.6	2025	Linux, MacOS	pytest	No
MutPy	Python	3.3 - 3.11	2019	Windows, Linux, MacOS	pytest, unittest	Yes
XMutant	Python	Python 3.	2018	Windows, Linux, MacOS	doctests	No
Mutatest	Python	Python 3.7 or 3.8.	2023	Windows, Linux, MacOS	pytest	Yes
Poodle	Python	Python 3.9 - 3.12	03/02/2024	Windows, Linux, MacOS	All relevant	No
	Selective mutating	Parallel mutating	Report format	Ease of setup	Community support	AST Parser
Cosmic Ray	Yes	Yes	Html, console, json database dump, XML	Moderate	Moderate	Parso
MutMut	Yes	Yes	custom GUI or .html	Moderate	Moderate	Parso
MutPy	Yes	No	YAML/HTML	Easy	Moderate	ast
XMutant	Yes	Yes	Text	Easy	None	bytecode
Mutatest	Yes	Yes	Text	Moderate	Moderate	ast
Poodle	Yes	Yes	Text, Html, Json	Moderate	Low	ast

Obr. 8: Porovnanie základných vlastností mutation testing frameworkov.

Mutation example	Mutation type	mutmut	mutpy	cosmic ray
a + b → a - b	Arithmetic Operator Replacement (AOR)			
x & y → x	Bitwise Operator Replacement (BOR)			
x == y → x != y	Comparison Operator Replacement (COR)			
and → or, True → False	Logical Operator Replacement (LOR)			
-x → +x	Unary Operator Replacement (UOR)			
break → continue	Control Flow Replacement (CFR)			
5 → 0	Numerical constant Replacement (CRP)			
raise A → raise B	Exception Handling Mutations (EXM)			
removes @decorator	Decorator Removal (DEC)			
x → y	Variable Replacement (VRP)			
"hello" → "world"	String mutator			
Deletes entire if block	Block statement mutator			
return x → return None	Method mutator			

Obr. 9: Porovnanie podpory mutačných operátorov.

Voľba frameworku – prečo Cosmic Ray

Na základe analýzy sme si vybrali framework **Cosmic Ray**, pretože:

- má aktívnu komunitu a pravidelné aktualizácie,
- podporuje všetky relevantné verzie Pythonu a všetky OS (Windows, Linux, macOS),
- funguje so všetkými bežne používanými testovacími frameworkami (pytest, unittest),
- poskytuje paralelné mutačné behy, selective mutation filtering a databázové výstupy

Tieto porovnávacie tabuľky sú univerzálne a môžu slúžiť aj iným vývojárom, ktorí hľadajú vhodný mutation testing framework pre svoj Python projekt. Ide o rýchly spôsob, ako porovnať kvalitu, funkčnosť a udržiateľnosť dostupných riešení bez potreby manuálneho testovania každého nástroja.

2.6.2 Odhalenie chýb v knižnici **validators** pomocou mutation testingu

Počas experimentov s mutation testingom sme aplikovali cielené mutačné testovanie aj na verejne používanú Python knižnicu **validators**, konkrétne jej modul **finance.py** (validátory ISIN, CUSIP, SEDOL). Napriek tomu, že knižnica obsahuje vlastné unit testy, mutácie odhalili viacero logických chýb v implementáciách týchto validátorov, ktoré pôvodné testy nedokázali zachytiť.

Najzasadnejším problémom bola nesprávna implementácia kontrolného súčtu (Luhn checksum) – nielen pri ISIN, ale aj pri súvisiacich validátoroch. Ako príklad uvádzame ISIN validátor, kde sa premenná **check** počas celého výpočtu vôbec nemení, čo spôsobí, že validátor akceptuje aj neplatné identifikátory.

Na obrázku nižšie sú dva hlavné indikátory problému: (i) vysoký počet preživších mutácií pri **finance** validátoroch a (ii) chybná implementácia, ktorá nikdy neaktualizuje hodnotu **check**.

Cosmic Ray Report

Summary info
Date time: 25/11/2025 01:01:13
Total jobs: 520
Complete: 520 (100.00%)
Surviving mutants: 180 (34.62%)

Obr. 10: Výsledok mutation testovania – vysoký podiel preživších mutácií vo **finance** validátoroch.

Na základe týchto výsledkov sme vytvorili oficiálne issue v repozitári knižnice a upozornili autorov na zistené problémy vrátane analýzy chybnej logiky a referencií na výsledky

```
def _isin_checksum(value: str): 1 usage  Ⓐ Jovial Joe Jayarson
    check, val = 0, None

    for idx in range(12):
        c = value[idx]
        if c >= "0" and c <= "9" and idx > 1:
            val = ord(c) - ord("0")
        elif c >= "A" and c <= "Z":
            val = 10 + ord(c) - ord("A")
        elif c >= "a" and c <= "z":
            val = 10 + ord(c) - ord("a")
        else:
            return False

    if idx & 1:
        val += val

    return (check % 10) == 0
```

Obr. 11: Príklad chyby v ISIN validátore – premenná `check` zostáva na hodnote 0.

mutation testingu [17]. Ide o praktickú ukážku, ako mutation testing dokáže odhaliť chyby, ktoré jednotkové testy nepostrehnú ani v etablovaných knižniciach.

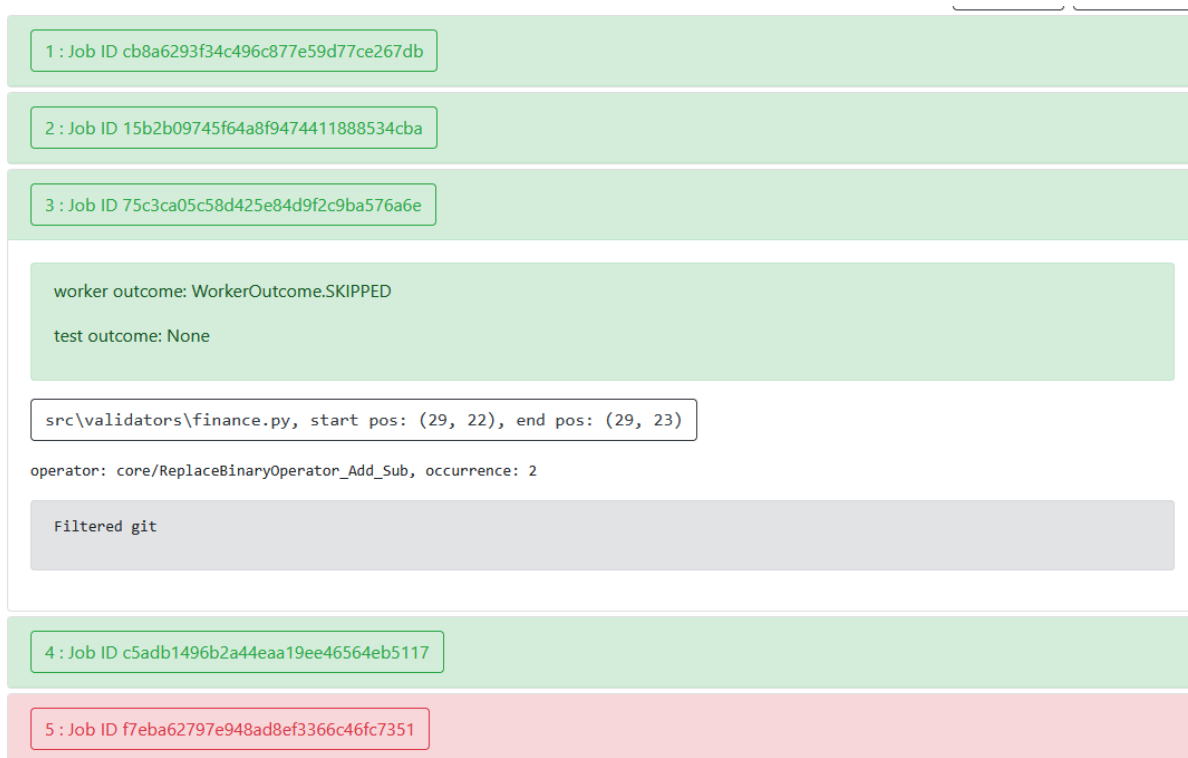
2.6.3 Úprava HTML reportov v Cosmic Ray

Súčasťou projektu bola aj analýza praktických problémov pri používaní frameworku Cosmic Ray v cielenom (targeted) mutation testovaní. Pri spusteniach s git-based filtrami, line-based mutátorom, operátorovými filtrami a ďalšími mechanizmami, ktoré sme v projekte používali, bolo množstvo mutácií správne preskočených ešte pred samotným vykonaním testov. Tieto mutácie však Cosmic Ray napriek tomu zobrazoval v HTML reporte.

V základnom zobrazení sa tieto položky vizuálne javili ako úspešne zabité mutácie, pričom informácia o tom, že išlo o mutácie označené ako `skipped`, bola viditeľná až po rozkliknutí detailu. Takéto správanie spôsobovalo výrazný vizuálny šum, skresľovalo mutation score a komplikovalo analýzu výsledkov pri targeted testovaní.

Na nasledujúcom obrázku je ukážka pôvodného stavu – mutácia je v zozname označená zelenou, hoci bola preskočená, a až v detaile je výsledok `WorkerOutcome.SKIPPED`:

Na odstránenie tohto problému bola do nástroja `cr-html` doplnená funkcionálna úplné skrytie mutácií so stavom `skipped`. Funkcionálna bola implementovaná priamo v oficiálnom kóde nástroja Cosmic Ray a bola navrhnutá na zaradenie do upstream verzie formou pull requestu [1]. Technické detaily implementácie vrátane úprav v generátore HTML reportov sú uvedené v sekcii 4.1.



Obr. 12: Pôvodné správanie HTML reportu – preskočené mutácie sa zobrazovali ako úspešné.

2.6.4 Zabezpečenie behu RooCode v PyCharme

Pre použitie AI asistenta RooCode v mutation-testing workflowoch bolo potrebné vyriešiť jeho integráciu v prostredí PyCharm. PyCharm štandardne podporuje iba AI asistentov (napr. JetBrains AI, Tabnine, CodeGPT), ktorí neposkytujú autonómiu potrebnú na komplexné workflowy – čítanie a úpravu súborov, vykonávanie príkazov či dlhodobé plánovanie krokov.

Ako riešenie bol otestovaný a zdokumentovaný nástroj **RunVSAgent**, ktorý funguje ako most medzi prostredím PyCharm a VS Code agentmi, vrátane RooCode. Toto riešenie umožňuje spúšťať RooCode priamo v PyCharme a využívať ho pri mutation-testing experimentoch aj pri návrhu a realizácii promptového workflowu.

Súčasťou projektu je aj samostatná dokumentácia, ktorá popisuje kompletný postup inštalácie a konfigurácie tohto riešenia. Dokumentácia je dostupná v repozitári projektu a umožňuje rovnakú konfiguráciu jednoducho zopakovať aj ďalším členom tímu alebo externým používateľom [16].

2.6.5 Rozšírenie kontextu mutácií o názov funkcie

Počas analýzy výstupov nástroja Cosmic Ray sme identifikovali nedostatok v reportovaní chýb. Pôvodná identifikácia miesta mutácie, založená iba na ceste k súboru a čísle riadku, neposkytovala dostatočný sémantický kontext. Pre rýchlu analýzu a filtráciu mutácií je kľúčové vedieť nielen to, na ktorom riadku mutácia nastala, ale aj to, akej logickej jednotke (funkcii, metóde alebo triede) daný kód patrí.

Navrhli sme preto rozšírenie funkcionality nástroja v dvoch rovinách:

- **Úprava dátového modelu:** Bolo potrebné rozšíriť databázovú schému o nový

atribút, ktorý by umožnil trvalé uloženie názvu funkcie priamo k záznamu o mutácii. Tým sa zabezpečilo, že kontext zostane zachovaný aj pre neskoršie generovanie reportov.

- **Algoritmus pre získanie kontextu:** Implementovali sme logiku založenú na prechádzaní abstraktného syntaktického stromu (AST). Algoritmus pri generovaní mutácie analyzuje hierarchiu uzlov – postupuje od mutovaného miesta smerom ku koreňu stromu (rodičovským uzlom), kým nenarazí na definíciu funkcie alebo triedy.

Toto vylepšenie umožňuje vo výsledných reportoch okamžite rozlíšiť, či sa mutácia nachádza v kritickej biznis logike alebo v menej podstatnej pomocnej metóde, čím sa výrazne zefektívňuje proces vyhodnocovania testov.

Výsledná podoba HTML reportu po zapracovaní zmien je znázornená na obrázku 13.

The screenshot displays a Cosmic Ray HTML report. At the top, three job IDs are listed in colored boxes: 159 (green), 160 (red), and 161 (red). Below these, a section titled 'SURVIVED' shows 'worker outcome: WorkerOutcome.NORMAL' and 'test outcome: TestOutcome.SURVIVED'. A code diff section shows a change in 'src\marshmallow\utils.py' at line 44, column 15. The diff highlights the function 'from_timestamp' with a green box. The code diff shows a change from a comment to a function definition.

```
159 : Job ID 59d35058ae294ae4b15f832a821b688a
160 : Job ID 8d856ded16814f68841e5b84c3d807ee
161 : Job ID 96453df54cba454f96717ccefada0f4d

SURVIVED
worker outcome: WorkerOutcome.NORMAL
test outcome: TestOutcome.SURVIVED

src\marshmallow\utils.py, start pos: (44, 15), end pos: (44, 16)
operator: core/NumberReplacer, occurrence: 1, function_name: from_timestamp

--- mutation diff ---
--- asrc\marshmallow\utils.py
+++ bsrc\marshmallow\utils.py
@@ -41,7 +41,7 @@
     if value is True or value is False:
         raise ValueError("Not a valid POSIX timestamp")
     value = float(value)
-    if value < 0:
+    if value < -1:
         raise ValueError("Not a valid POSIX timestamp")

# Load a timestamp with utc as timezone to prevent using system timezone.
```

Obr. 13: Výsledný HTML report v Cosmic Ray obsahujúci kontext názvu funkcie.

Toto rozšírenie sme realizovali v rámci zdrojového kódu nástroja a zmeny boli navrhnuté na integráciu do oficiálnej verzie projektu prostredníctvom pull requestu [2]. Technické detaily implementácie vrátane úprav dátových štruktúr a algoritmu sú uvedené v sekcii 4.2.

3 Technická dokumentácia

3.1 Použité technológie

3.1.1 Programovací jazyk

V projekte bol použitý programovací jazyk Python. Slúžil ako hlavný jazyk pre spúšťanie mutation testing frameworkov, tvorbu pomocných skriptov a spracovanie výsledkov experimentov. Python bol zvolený pre svoju čitateľnosť, širokú podporu testovacích nástrojov a vhodnosť pre experimentálne a výskumné projekty.

3.1.2 Frameworky pre mutation testing

V rámci projektu boli použité, testované a porovnávané viaceré frameworky pre mutation testing v jazyku Python. Tieto nástroje umožňujú generovanie mutácií v zdrojovom kóde s cieľom vyhodnotiť kvalitu a efektívnosť testovacích sád. Konkrétne boli použité nasledovné frameworky:

- Cosmic Ray
- MutMut
- MutPy
- XMutant
- Mutatest
- Poodle
- Stryker

Jednotlivé frameworky [3] boli analyzované z hľadiska podporovaných mutačných operátorov, compatibility s verziami jazyka Python, výkonnosti a prehľadnosti výstupov.

3.1.3 Vývojové prostredia

Na vývoj a experimentovanie boli použité vývojové prostredia Visual Studio Code a PyCharm. Obe prostredia poskytujú podporu pre jazyk Python, ladenie kódu, spúšťanie testov a integráciu so systémom správy verzií. Použitie viacerých vývojových prostredí umožnilo flexibilnejšiu prácu počas vývoja a hodnotenia mutation testing frameworkov.

3.1.4 Roo Code

Nástroj Roo Code bol využívaný v kombinácii s vývojovými prostrediami Visual Studio Code a PyCharm. Prostredníctvom príkazov Roo Code bola podporená automatizácia úloh, generovanie kódu a analýza experimentov súvisiacich s mutation testingom. Použitie tohto nástroja prispelo k zvýšeniu efektivity práce a lepšej reprodukovateľnosti výsledkov.

3.1.5 Databáza

Na ukladanie výsledkov experimentov a údajov získaných z mutation testingu bola použitá databáza SQLite3. Ide o ľahkú relačnú databázu, ktorá funguje lokálne a nevyžaduje samostatný databázový server. SQLite3 bola zvolená pre svoju jednoduchosť, nízke nároky na konfiguráciu a vhodnosť pre experimentálne projekty.

3.1.6 Správa verzií

Na správu zdrojového kódu bol použitý systém Git. Vzdialené úložisko projektu je hostované na platforme GitHub, ktorá umožňuje sledovanie zmien, prácu s vetvami a prehľadnú organizáciu jednotlivých experimentov a porovnaní mutation testing frameworkov.

3.2 Deployment diagram

Na obrázku 14 je znázornená architektúra riešenia z pohľadu nasadenia. Diagram zachytáva všetky, ktoré vstupujú do procesu mutačného testovania riadeného AI agentom RooCode, ako aj ich vzájomné väzby spolu s prostrediami, v ktorých jednotlivé časti systému bežia.

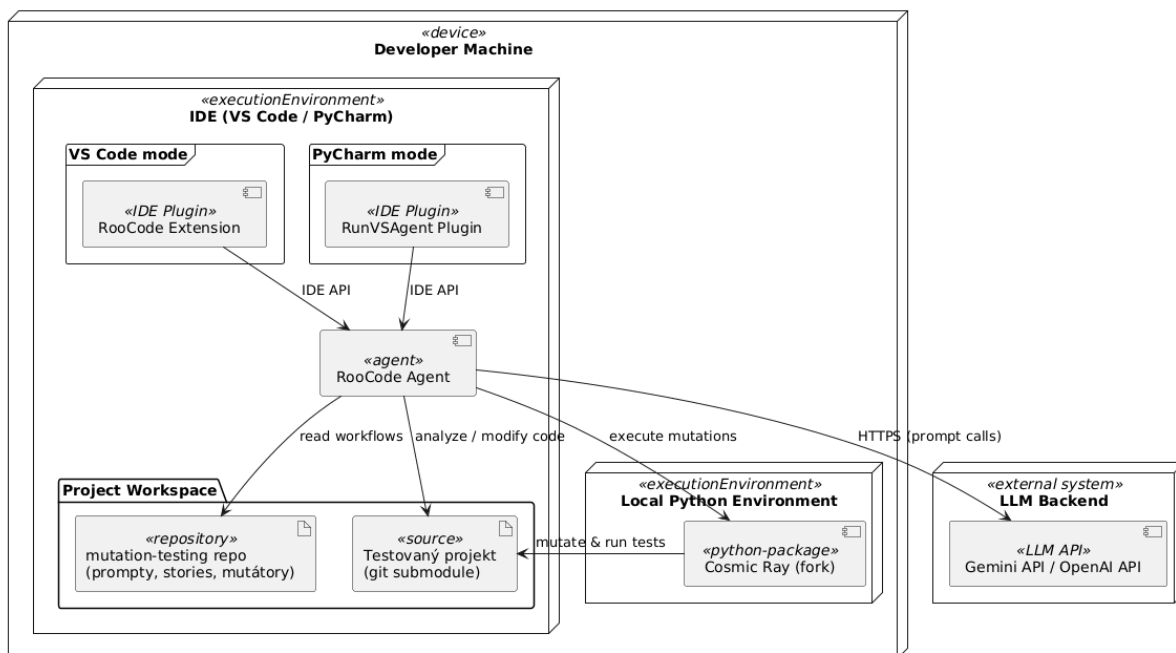
Celý systém je nasadený na vývojárskom počítači, ktorý predstavuje primárne spúšťacie prostredie pre AI workflow, testovanie aj samotné mutačné behy. Vývojár môže pracovať v dvoch podporovaných prostrediach: *VS Code* alebo *PyCharm*. V prípade VS Code je RooCode spúšťaný natívne cez rozšírenie **RooCode Extension**, zatiaľ čo v prostredí PyCharm je použitý plugin **RunVSAgent**, ktorý sprostredkuje rovnaké API a umožňuje PyCharmu komunikovať s RooCode agentom prostredníctvom VS Code protokolu. V oboch prípadoch je výsledkom jeden spoločný komponent – **RooCode Agent** – ktorý vykonáva celý workflow riadeného mutation testovania.

Agent má prístup k dvom hlavným častiam pracovného priestoru projektu. Prvou je repositár *mutation-testing*, ktorý obsahuje pripravované .md prompty, user stories a vlastné mutátory, ktoré definujú jednotlivé kroky AI workflowu. Druhou časťou je *testovaný projekt*, zvyčajne pripojený ako git submodule, nad ktorým sa vykonáva cieľené mutation testovanie. RooCode agent tieto zdroje číta, analyzuje a podľa potreby upravuje.

Mutačné testovanie je realizované v lokálnom Python prostredí, kde je nainštalovaná upravená, projektom rozšírená verzia nástroja **Cosmic Ray**. RooCode agent komunikuje priamo s touto inštaláciou a spúšťa mutačné behy nad testovaným projektom. Cosmic Ray následne vykonáva mutácie, spúšťa testy a vracia späť dáta o zabitých, preživších a preskočených mutáciách.

Externým komponentom je *LLM backend*, ktorý predstavuje vzdialenú inferenčnú službu (Gemini API / OpenAI API). RooCode agent s ňou komunikuje cez HTTPS a využíva ju na generovanie plánov, výber mutátorov, vyhodnocovanie výsledkov a ďalšie kroky vyplývajúce z promptového workflowu.

Diagram tak poskytuje pohľad na to, ako je celý systém zložený: od AI asistenta, cez vývojové prostredie a lokálne nástroje až po externý LLM backend, ktoré spolu umožňujú vykonávanie cieľného mutation testovania v Python projektoch.



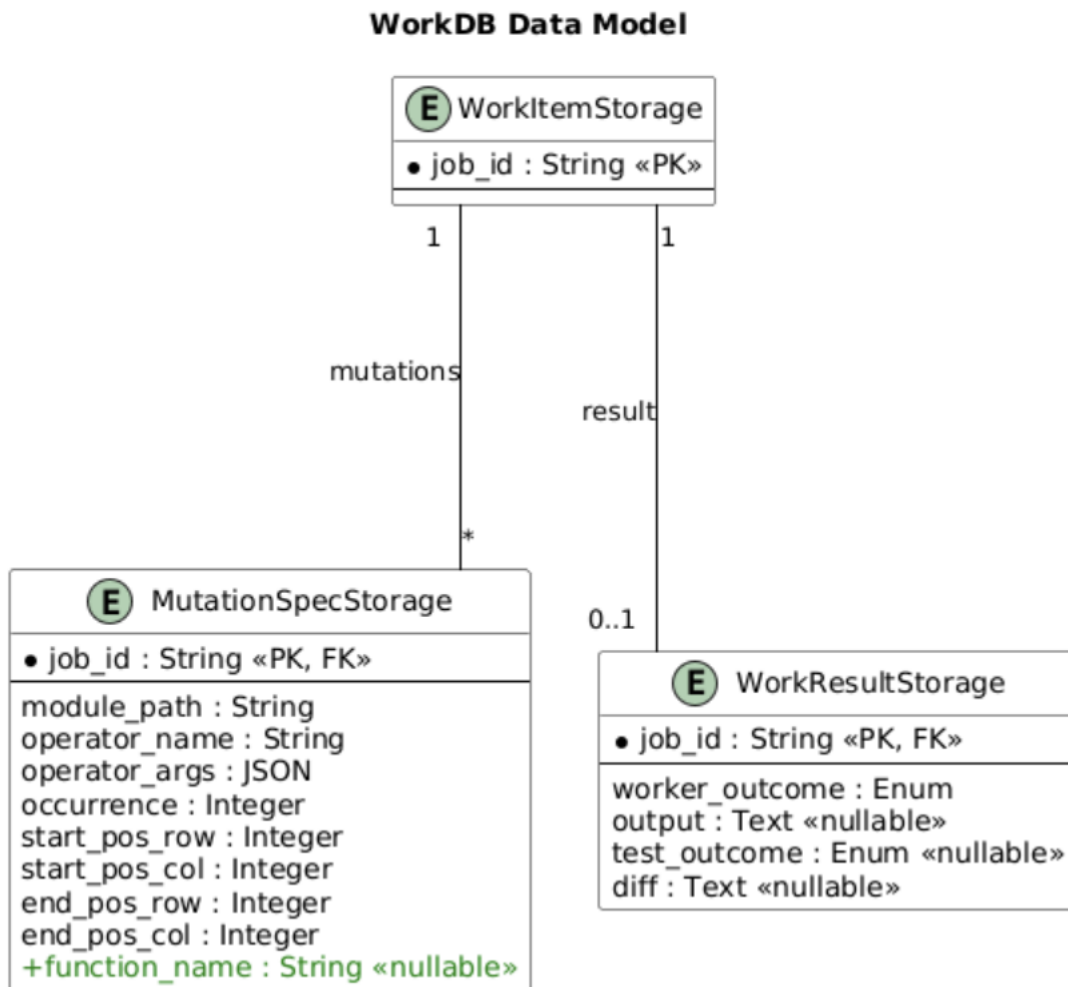
Obr. 14: Deployment diagram riešenia mutation-testingu riadeného AI agentom RooCode.

3.3 Dátový model

Perzistencia údajov v nástroji Cosmic Ray je zabezpečená relačnou databázou SQLite. Dátový model je definovaný pomocou ORM knižnice SQLAlchemy a pozostáva z troch hlavných entít, ktoré sú navzájom prepojené prostredníctvom unikátneho identifikátora úlohy (`job_id`).

Schéma databázy (Obrázok 15) obsahuje nasledovné entity:

- **WorkItemStorage**: Predstavuje centrálnu evidenciu naplánovanej úlohy (Job).
- **MutationSpecStorage**: Obsahuje detailnú definíciu mutácie (napr. cestu k súboru, typ operátora). S entitou **WorkItem** má väzbu **1:1**. V tejto tabuľke sme realizovali **rozšírenie dátového modelu** o atribút `function_name` 4.2, ktorý uchováva názov funkcie pre zachovanie kontextu chyby.
- **WorkResultStorage**: Slúži na uloženie výsledku testu (výstup, diff). S entitou **WorkItem** má väzbu typu **1:0..1**.



Obr. 15: ER diagram dátového modelu Cosmic Ray s vyznačeným rozšírením

4 Zaujímavé fragmenty implementácie

Táto kapitola prezentuje vybrané technické fragmenty a implementačné úpravy, ktoré vznikli počas riešenia projektu. Ide o praktické zásahy do existujúcich nástrojov, konfiguračných súborov a pracovných postupov, ako aj o návrh špecializovaných promptov pre AI asistenta RooCode. Tieto fragmenty vznikli ako reakcia na konkrétne problémy identifikované pri experimentoch s mutation testingom v reálnych Python projektoch.

Časť prezentovaných ukážok sa týka úprav zdrojového kódu existujúcich mutation testing frameworkov (napr. CosmicRay), ktoré bolo potrebné rozšíriť alebo prispôbiť tak, aby poskytovali prehľadnejšie výstupy a lepšie podporovali cielené testovacie scenáre. Ďalšie fragmenty reprezentujú automatizačné postupy vo forme promptov alebo jednoduchých skriptov, ktoré demonštrujú, ako je možné pomocou AI asistenta riadiť inštaláciu, konfiguráciu a spúšťanie mutation testov bez potreby manuálneho zásahu do každého kroku procesu.

Spoločným cieľom všetkých uvedených fragmentov je zvýšiť praktickú použiteľnosť mutation testingu, znížiť zbytočný šum vo výstupoch nástrojov a ukázať, ako je možné existujúce riešenia efektívne kombinovať s automatizáciou a AI asistovaným workflowom.

4.1 Úprava HTML reportov v Cosmic Ray: skrytie preskočených mutácií

Táto časť dopĺňa analytický popis úpravy HTML reportov uvedený v sekcii 2.6.3 a zameriava sa na ukážku implementačných detailov.

Na úrovni príkazového rozhrania bol nástroj `cr-html` rozšírený o nový prepínač `-hide-skipped`, ktorý umožňuje používateľovi explicitne vylúčiť preskočené mutácie z výsledného HTML reportu. Obrázok 16 ilustruje rozšírenie deklarácie CLI príkazu a spôsob, akým je hodnota parametra propagovaná do generátora reportu.

```
@click.command()  ⚡ Ondrej Babinský +2
@click.option(*param_decls: "--only-completed/--not-only-completed", default=False)
@click.option(*param_decls: "--skip-success/--include-success", default=False)
@click.option(*param_decls: "--hide-skipped/--show-skipped", default=False)
@click.argument(*param_decls: "session-file", type=click.Path(dir_okay=False, readable=True, exists=True))
def report_html(only_completed, skip_success, hide_skipped, session_file):
    """Print an HTML formatted report of test results."""
    with use_db(session_file, WorkDB.Mode.open) as db:
        doc = _generate_html_report(db, only_completed, skip_success, hide_skipped)
```

Obr. 16: Rozšírenie príkazu `cr-html` o prepínač `-hide-skipped`.

Samotné filtrovanie mutácií prebieha priamo počas generovania jednotlivých položiek reportu. Ako je znázornené na obrázku 17, mutácie so stavom `worker_outcome = "skipped"` sú pri zapnutom prepínači úplne vynechané ešte pred vykreslením HTML prvku. Tým sa zabezpečí, že sa tieto mutácie v reporte vôbec neobjavia a neovplyvňujú vizuálnu interpretáciu výsledkov.

```
def _generate_work_item_card(doc, index, work_item, result, skip_success, hide_skipped):
    doc, tag, text = doc.tagtext()
    if hide_skipped and result is not None and result.worker_outcome == "skipped":
        return
```

Obr. 17: Podmienkové vynechanie mutácií so stavom `skipped` pri generovaní reportu.

Táto úprava predstavuje malý, no prakticky významný zásah do kódu frameworku Cosmic Ray, ktorý zlepšuje čitateľnosť reportov najmä pri cielenom mutation testovaní s rozsiahlym predbežným filtrovaním mutácií.

4.2 Rozšírenie kontextu mutácií o názov funkcie

Táto sekcia dopĺňa analytický popis problému chýbajúceho kontextu uvedeného v sekcii 2.6.5 a zameriava sa na ukážku implementačných detailov.

Pre zabezpečenie perzistencie názvu funkcie bolo potrebné upraviť interné dátové štruktúry nástroja. Obrázky 18 a 19 ilustrujú rozšírenie ORM modelu `MutationSpecStorage` aj dátovej triedy `MutationSpec` o nový atribút `function_name`.

```

class MutationSpecStorage(Base):
    "Database model for MutationSpecs"

    __tablename__ = "mutation_specs"
    module_path = Column(String)
    operator_name = Column(String)
    operator_args = Column(JSON)
    occurrence = Column(Integer)
    start_pos_row = Column(Integer)
    start_pos_col = Column(Integer)
    end_pos_row = Column(Integer)
    end_pos_col = Column(Integer)
    function_name = Column(String, nullable=True)
    job_id = Column(String, ForeignKey("work_items.job_id"), primary_key=True)
    work_item = relationship("WorkItemStorage", back_populates="mutations")

```

Obr. 18: Rozšírenie databázového modelu MutationSpecStorage o nový stĺpec.

```

class MutationSpec:
    "Description of a single mutation."

    module_path: Path = field(converter=Path)
    operator_name: str = field()
    occurrence: int = field(converter=int)
    start_pos: tuple[int, int] = field()
    end_pos: tuple[int, int] = field()
    operator_args: dict[str, Any] = field(factory=dict)
    function_name: Optional[str] = field(default=None)

```

Obr. 19: Pridanie atribútu function_name do triedy MutationSpec.

Samotné získavanie kontextu zabezpečuje nová metóda `get_definition_name`. Ako je znázornené na obrázku 20, algoritmus prechádza abstraktným syntaktickým stromom (AST) od miesta mutácie smerom k rodičovským uzlom. Ak narazí na uzol typu `FunctionDef` alebo `AsyncFunctionDef`, vráti jeho názov, čím identifikuje logický blok kódu.

```

def get_definition_name(self):
    "Get the name of the function or class enclosing the current node."
    obj = self.obj
    while obj:
        if isinstance(obj, (parso.python.tree.Function, parso.python.tree.Class)):
            return obj.name.value
        obj = obj.parent
    return None

```

Obr. 20: Implementácia vyhľadávania nadradenej definície v AST strome.

Vďaka týmto úpravám je názov funkcie perzistentne uložený priamo pri zázname mutácie a pripravený na zobrazenie vo výslednom reporte.

4.3 Automatická AI analýza preživších mutácií

Po vykonaní mutation testingu zostala množina mutácií označených ako *survived*. Ich analýza a klasifikácia bola vykonaná plne automatizovane pomocou nástroja **roo code**, ktorý využíva umelú inteligenciu na vyhodnocovanie výsledkov mutation testingu bez manuálneho zásahu používateľa.

Cieľom tejto fázy bolo určiť príčinu prežitia každej mutácie a zároveň navrhnúť konkrétny krok, ktorým je možné danú mutáciu v budúcnosti eliminovať.

4.3.1 Ukážka časti **roo code** scenára pre vyhodnotenie mutation testingu

V rámci nástroja **roo code** je proces vyhodnotenia mutation testingu definovaný ako formálny scenár (use case)[11, 12, 13], ktorý určuje preconditions, postup spracovania a očakávané výstupy. Nejde teda o statickú „šablónu“ reportu, ale o časť „kódu“ (špecifikácie scenára), ktorá riadi generovanie finálnych artefaktov: interpretáciu výsledkov zo session súboru, vytvorenie reportov a následnú analýzu mutácií.

Preconditions

- A project has been set up with mutation testing capabilities.
- Mutation tests have been executed, and a session file (e.g., `session.sqlite`) containing the results is available.

Flow

1. Clean and Verify Codebase:

- Agent follows the steps outlined in **Use Case 04.1 - Pre-evaluation Cleanup** to clean and verify the codebase. This includes running linting, static analysis, and code coverage.

2. Generate and Interpret Reports:

- Agent follows the steps outlined in **Use Case 04.2 - Generate and Interpret Report** to generate structured mutation testing reports and interpret their outcomes. This includes generating an HTML report, and creating markdown files for survived and killed mutants.

3. Analyze and Classify Mutants:

- Agent analyzes the survived and killed mutants, providing reasoning and suggesting fixes for each.

Postconditions

- The codebase is cleaned and verified for evaluation.
- Structured mutation testing reports are generated and interpreted.
- Weak spots in the test suite or code are identified.

Obr. 21: Ukážka časti scenára v 04-evaluation-mutation-tests, ktorý definuje tok vyhodnotenia mutation testingu a generovanie výsledných výstupov.

Takto definovaný postup zabezpečuje opakovateľnosť spracovania a zároveň poskytuje transparentný popis toho, aké kroky AI systém vykonáva pri tvorbe finálnych výstupov.

4.3.2 Princíp AI hodnotenia

AI systém pracuje s kontextom celej aplikácie a analyzuje:

- rozdiel medzi pôvodným a zmutovaným kódom (mutation diff),
- zdrojový kód v mieste mutácie,
- existujúce testovacie prípady a ich pokrytie,

- správanie testov pri behu mutácie.

Na základe tejto analýzy je každá mutácia automaticky zaradená do jednej z kategórií *testing error*, *coding error* alebo *unknown* a zároveň je vygenerovaný návrh riešenia.

4.3.3 Výstup AI analýzy

Výsledky analýzy sú ukladané do prehľadných výstupných súborov, ktoré obsahujú presnú lokalizáciu mutácie, použitý mutačný operátor, dôvod prežitia a odporúčanú úpravu. Typickým príkladom je mutácia, pri ktorej AI identifikovala nedostatočné testovanie porovnania singleton objektov a navrhla doplnenie chýbajúceho testu.

src\validators_extremes.py

- **Operator:** core/ReplaceComparisonOperator_IsNot_NotEq
- **Line:** 47
- **Diff:**

```
--- mutation diff ---
--- asrc\validators\_extremes.py
+++ bsrc\validators\_extremes.py
@@ -44,5 +44,5 @@
def __le__(self, other: Any):
    """LessThanOrEqual."""
-   return other is not AbsMin
+   return other != AbsMin
```

- **Classification:** Fix tests
- **Reasoning:** The mutant survived because `is not` and `!=` are equivalent for singletons like `AbsMin`. The tests do not cover a scenario where a different instance of `AbsMin` is created, which would cause the `!=` operator to behave differently.
- **Fix:** Add a test case that creates a new instance of `AbsMin` and compares it to the original.

```
def test_abs_min_is_not_equal_to_new_instance():
    assert AbsMin() != AbsMin()
```

Obr. 22: Ukážka výstupu AI analýzy preživšej mutácie vrátane klasifikácie a návrhu riešenia.

4.4 Implementácia custom mutátora

```
1 from cosmic_ray.operators.operator import Operator
2 from parso.python.tree import ReturnStmt, YieldExpr
3 from parso import parse
4
5 class ShortenTuple(Operator):
6
7     def mutation_positions(self, node):
8         if isinstance(node, (ReturnStmt, YieldExpr)) and len(node.children) >= 2:
9             return_value = node.children[1]
10             if return_value.type == "atom" and len(return_value.children) == 3:
11                 left_operator = return_value.children[0]
12                 tuple_values = return_value.children[1]
13                 right_operator = return_value.children[2]
14                 if (
15                     left_operator.type == "operator" and left_operator.value == "("
16                     and tuple_values.type == "testlist_comp" and len(tuple_values.children) >= 2
17                     and right_operator.type == "operator" and right_operator.value == ")"
18                 ):
19                     yield (tuple_values.start_pos, tuple_values.end_pos)
20
21     def mutate(self, node, index):
22         tuple_values = node.children[1].children[1]
23         if len(tuple_values.children) == 3:
24             tuple_values.children = tuple_values.children[:-1]
25         else:
26             tuple_values.children = tuple_values.children[:2]
27         return node
28
29     def examples(self):
30         return [(parse('return (1, 2, 3)').children[0], parse('return (1, 2)').children[0])]
```

Obr. 23: Ukážka implemtácie tuple shortener custom mutátora

Cosmic Ray používa parsovaciu knižnicu parso, ktorá pužíva AST na reprezentáciu kódu. Trieda, ktorá implementuje custom mutátora, musí mať nasledujúce tri funkcie:

1. Funkcia `mutation_positons` nájde každú node, kde sa má vykonať custom mutácia, a vracia začiatočnú a konečnú pozíciu všetkých takýchto node.
2. Funkcia `mutate` vykoná implementovanú mutáciu na node, ktorá bola nájdená pomocou funkcie `mutation_positons`. Ak sa má daná node odstrániť, tak táto funkcia vracia hodnotu `None`.
3. Funkcia `examples` popisuje zmenu kódu pred a po mutácií, pričom vracia list n-tíc (tuples), ktoré obsahujú 2 prvky, prvý prvok je príklad kódu pred mutáciou a druhý prvok je zmena prvého prvku po mutácií.

5 Inštalačná príručka

Táto kapitola popisuje kompletný postup sprevádzkovania prostredia pre mutačné testovanie. Celý proces je navrhnutý tak, aby zabezpečil izoláciu testovaného projektu a konzistentné použitie nástrojov.

Postup je rozdelený do dvoch hlavných fáz: príprava pracovného priestoru (podľa Use Case 01 [6]) a následná integrácia mutačného nástroja do cieľového projektu (podľa Use Case 01.1 [7]).

5.1 Príprava pracovného priestoru

Prvým krokom je príprava tohto repozitára a následné vloženie cieľového projektu určeného na testovanie.

1. **Klonovanie repozitára:** Stiahnite si tento projekt na lokálny disk a presuňte sa do jeho adresára:

```
git clone https://github.com/Bolest-oci/mutation-testing.git
cd mutation-testing
```

2. **Pridanie testovaného projektu:** Cieľový projekt pridajte ako git submodule do adresára `repos/`. Namiesto `<repository_url>` dosadte adresu vášho repozitára a namiesto `<project_name>` zvolte názov priečinka:

```
git submodule add <repository_url> repos/<project_name>
```

5.2 Inštalácia nástroja Cosmic Ray

Pre využitie implementovaných rozšírení (popísaných v sekciách 4.2 a 4.1) je nutné inštalovať nástroj z nášho upraveného forku. Inštalačný proces zahŕňa detekciu používaného nástroja na správu závislostí (na základe prítomnosti konfiguračných súborov), podľa čoho sa zvolí adekvátny príkaz na inštaláciu cez protokol `git+https`.

Príklad inštalácie pre prostredie využívajúce štandardný `pip`:

```
pip install git+https://github.com/RichardMacko/cosmic-ray.git
```

5.3 Synchronizácia a overenie

V závislosti od zvoleného nástroja môže byť následne vyžadovaná synchronizácia prostredia (napr. uv `sync`, `poetry install` alebo `pdm sync`) na aplikovanie zmien.

Na záver overte dostupnosť nástroja príkazom:

```
pip show cosmic-ray
```

Výstup, najmä položka **Location**, musí potvrdzovať, že balík je nainštalovaný v rámci virtuálneho prostredia daného projektu.

6 Používateľská príručka

Proces mutation-based testovania bol v projekte automatizovaný pomocou agenta RooCode, ktorý slúži ako hlavné rozhranie medzi používateľom a nástrojom Cosmic Ray. Cieľom tohto prístupu je abstrahovať technické detaily jednotlivých krokov a umožniť používateľovi pracovať so systémom na vyššej úrovni.

RooCode nefunguje ako jednoduchý skript, ale ako riadený agent, ktorý sa pri vykonávaní jednotlivých operácií opiera o vopred definované používateľské príbehy (user stories) a prípady použitia (use cases) [14, 15]. Tieto špecifikácie slúžia ako formálny popis očakávaného správania systému a určujú, aké kroky má agent vykonať v jednotlivých fázach práce.

Používateľ zadáva len základné vstupy, ako je výber projektu alebo spustenie konkrétnej fázy testovania, zatiaľ čo RooCode na základe príslušných user stories a use cases zabezpečuje prípravu prostredia, konfiguráciu nástroja, vykonanie mutation testov a spracovanie výsledkov. Funkcionalita systému je rozdelená do logických oblastí, ktoré zodpovedajú jednotlivým používateľským príbehom.

Project and framework setup

Táto časť systému zabezpečuje úvodnú prípravu testovaného projektu. RooCode v tejto fáze postupuje podľa scenárov definovaných v use cases zameraných na nastavenie projektu a frameworku. Používateľ poskytne URL repozitára, ktorý je automaticky pripravený na mutation testovanie v izolovanom prostredí.

Agent vytvorí samostatný pracovný branch a nakonfiguruje všetky potrebné závislosti tak, aby bol projekt pripravený na ďalšie kroky. Použitie tejto funkcionality je plne automatizované a slúži ako vstupný bod práce so systémom.

Integration of custom mutators

Integrácia vlastných mutátorov je riadená user stories a use cases, ktoré popisujú spôsob rozšírenia mutation testovania. RooCode na ich základe identifikuje dostupné mutátory, zabezpečí ich korektnú konfiguráciu a zapojenie do nástroja Cosmic Ray.

Používateľ zvolí, ktoré rozšírenia majú byť použité, pričom samotné technické kroky úpravy konfigurácie a validácie nastavení sú vykonané automaticky agentom. Týmto spôsobom je možné testovať aj doménovo-špecifické alebo netypické chyby bez manuálnych zásahov do konfiguračných súborov.

Targeted mutation test runs

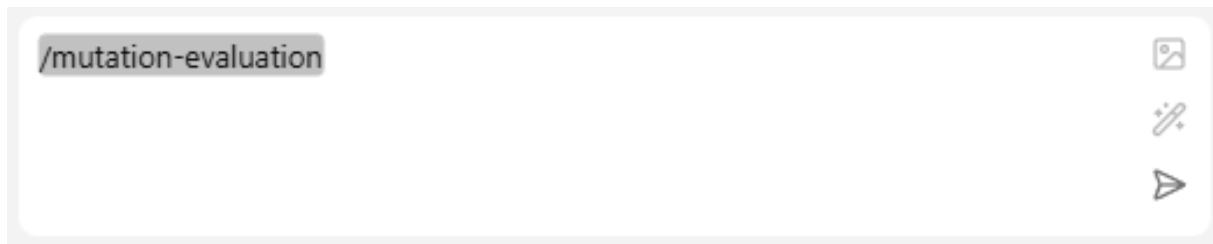
V oblasti cieleného mutation testovania RooCode využíva user stories zamerané na optimalizáciu rozsahu testovania. Na ich základe agent automaticky identifikuje zmeny v zdrojovom kóde a prispôsobí rozsah mutation testovania tak, aby sa testy vykonávali len nad relevantnými časťami projektu.

Používateľ tak pracuje s mutation testingom na úrovni vývojového workflow, zatiaľ čo RooCode zabezpečuje aplikáciu filtrov, výber mutačných operátorov a relevantných testov podľa definovaných prípadov použitia.

Mutation test evaluation

Vyhodnotenie výsledkov mutation testovania je riadené user stories a use cases zameranými na analýzu efektivity testov. RooCode na ich základe generuje prehľadné reporty a identifikuje oblasti, v ktorých existujúce testy nedokázali odhaliť chyby v zdrojovom kóde.

Výstupy sú štruktúrované tak, aby používateľ získal jasný a interpretovateľný prehľad o kvalite testovacej sady a o miestach vhodných na zlepšenie.



Obr. 24: Ukážka RooCode commandu na spustenie evaluácie

Test fixes and improvements

Záverečná časť systému vychádza z user stories zameraných na zlepšovanie testovacej sady. Na základe výsledkov predchádzajúcej analýzy RooCode navrhuje úpravy existujúcich testov alebo doplnenie nových prípadov, pričom rešpektuje definované use cases pre aplikáciu a overenie opráv.

Používateľ má možnosť navrhované zmeny schváliť a nechať ich automaticky aplikovať. Následné overenie potvrdzuje, že vykonané úpravy viedli k zlepšeniu mutation score a k zvýšeniu celkovej kvality testov.

Literatúra a odkazy

- [1] Cosmic ray pull request: Add `--hide-skipped` option to html report. <https://github.com/sixty-north/cosmic-ray/pull/573>. Accessed: 20 Jan 2026.
- [2] Cosmic ray pull request: feat(core): track and report function names for mutations. <https://github.com/sixty-north/cosmic-ray/pull/577>. Accessed: 20 Jan 2026.
- [3] Mutation frameworks xlxs. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/research/Mut-testing-frameworks-comparison.xlsx>. Accessed: 19 Jan 2026.
- [4] Mutation testing frameworks comparison — research tables. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/research/Mut-testing-frameworks-comparison.xlsx>. Accessed: 15 Jan 2026.
- [5] Run targeted mutation test runs prompt. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/03-run-targeted-mutation-test-runs.md>. Accessed: 15 Jan 2026.
- [6] Use case 01 - project setup. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/01-project-setup.md>. Accessed: 19 Jan 2026.
- [7] Use case 01.1 - install cosmic ray. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/01.1-install-cosmic-ray.md>. Accessed: 18 Jan 2026.
- [8] Use case 01.2 - run basic test. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/01.2-run-basic-test.md>. Accessed: 19 Jan 2026.
- [9] Use case 01.2.1 - run basic test - local. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/01.2.1-run-basic-test-local.md>. Accessed: 19 Jan 2026.
- [10] Use case 01.2.2 - run basic test - http. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/01.2.2-run-basic-test-http.md>. Accessed: 19 Jan 2026.
- [11] Use case 04 - evaluate mutation tests. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/04-evaluate-mutation-tests.md>. Accessed: 19 Jan 2026.
- [12] Use case 04.1 - pre-evaluation cleanup. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/04.1-pre-evaluation-cleanup.md>. Accessed: 19 Jan 2026.
- [13] Use case 04.2 - generate and interpret mutation testing results. <https://github.com/Bolest-oci/mutation-testing/blob/main/doc/use-cases/04.2-generate-and-interpret-report.md>. Accessed: 19 Jan 2026.
- [14] Use cases. <https://github.com/Bolest-oci/mutation-testing/tree/main/doc/use-cases>. Accessed: 19 Jan 2026.

- [15] User stories. <https://github.com/Bolest-oci/mutation-testing/tree/main/doc/user-stories>. Accessed: 19 Jan 2026.
- [16] Ondrej Babinský. Setup roocode in pycharm. <https://github.com/Bolest-oci/mutation-testing/wiki/Setup-RooCode-in-PyCharm>. Accessed: 15 Jan 2026.
- [17] python-validators maintainers. Incorrect finance validation logic. <https://github.com/python-validators/validators/issues/440>, 2025. Accessed: 2025-11-25.